

# GPAS: OPTIMIZER-AWARE TASK ALLOCATION FOR MULTI-TEACHER ON-POLICY DISTILLATION

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Multi-teacher on-policy distillation (MOPD) uses several specialist teachers to train one student. Existing task mixtures often combine two choices: the importance of each task in the training objective and the compute assigned to that task. We separate these choices and keep a per-response teacher-loss objective fixed. We then introduce Gradient-Preconditioned Adaptive Sampling (GPAS), which samples tasks using gradient scale after the current optimizer’s diagonal scaling. Cost-GPAS also uses the measured critical-path time of each teacher task. A task-wise AdamW second moment keeps the local optimizer geometry comparable when task probabilities or the number of tasks per update change. Controlled experiments verify the estimator calculations: GPAS reduces AdamW-scaled error by 28.3% relative to uniform sampling, and Cost-GPAS reduces the time-weighted error by 11.1% in the same construction. We connect each theoretical prediction to a four-teacher MOPD measurement at task widths  $K \in \{1, 2, 4\}$ . The evaluation uses matched response budgets, end-to-end GPU hours, teacher-switch traces, and conventional AdamW baselines. This design tests when adaptive task allocation improves MOPD efficiency under a fixed learning objective.

## 1 INTRODUCTION

Multi-teacher on-policy distillation (MOPD) uses specialist teachers to train a single student on several domains (Ma et al., 2026). A task mixture usually specifies both the learning objective and the share of training compute. An adaptive schedule can therefore change what the student learns while changing where compute is spent. Differences in teacher-loss scale, response length, and serving time make the two effects difficult to separate (Gao et al., 2026; Sun et al., 2026).

We study a focused question: given a fixed MOPD objective, how should a training run allocate teacher tasks? We separate an objective weight from a task’s inclusion probability. Importance correction preserves the weighted gradient when the execution policy changes. We also treat the number of tasks in one optimizer update, denoted by  $K$ , as a system and optimization choice. A small  $K$  permits teacher reuse and frequent updates. A large  $K$  covers more tasks per update and reduces task-selection noise; full coverage removes it.

Our method, Gradient-Preconditioned Adaptive Sampling (GPAS), measures each task’s gradient after the diagonal scaling used by the current optimizer. It assigns more task units to gradients with larger scaled magnitude. Cost-GPAS also favors tasks with shorter measured critical paths. Both rules leave the declared task objective unchanged. Figure 1 gives a controlled example in which raw gradient norms select the wrong tasks for AdamW, while GPAS reduces error in AdamW’s local geometry.

Adaptive task sampling also changes the squared gradients seen by conventional AdamW. We use a taskwise second-moment observation whose conditional mean stays fixed when the task proposal changes. This creates a common local AdamW geometry for comparing Uniform, GPAS, and Cost-GPAS. Conventional AdamW remains an experimental baseline because the two moment definitions can produce different training paths.

The paper makes three contributions:

- We define MOPD task allocation as an execution policy under a fixed per-response teacher-loss objective, with exact importance correction for updates containing  $K$  tasks.
- We propose GPAS and Cost-GPAS, two simple allocation rules based on optimizer-scaled gradient magnitude and measured task time. We give each formula a direct frozen-checkpoint or end-to-end test.
- We provide controlled evidence for the estimator and AdamW moment calculations, together with a four-teacher evaluation at  $K \in \{1, 2, 4\}$  that measures loss per response, loss per GPU hour, memory, and teacher switching.

## 2 RELATED WORK

**Task weighting and data allocation.** Multi-task methods tune loss weights, alter gradient directions, or adapt data schedules (Sener & Koltun, 2019; Yu et al., 2020; Liu et al., 2024; Chen et al., 2018). Carefully tuned scalarization is competitive with specialized multi-task methods in several settings (Kurin et al., 2022; Xin et al., 2022). Language-model training also adapts domain, rollout, or prompt distributions using learning progress (Albalak et al., 2023; Wang et al., 2025; Zheng et al., 2025; Gao et al., 2025). GPAS keeps task importance fixed and changes the data and compute allocation.

**Multi-teacher on-policy distillation.** MOPD routes student rollouts to task teachers and trains a shared student with dense token feedback (Ma et al., 2026). Open-MOPD studies unequal token budgets, reward scales, and stale rewards (Gao et al., 2026). D<sup>3</sup>-MOPD updates the domain mixture using teacher-loss gap and descent velocity (Sun et al., 2026). Teacher mixtures can also create transfer and task trade-offs (Li et al., 2026). These methods motivate reporting downstream capability separately from the fixed teacher-loss objective used in our study.

**Importance sampling for optimization.** Gradient-norm sampling reduces the second moment of an SGD estimate (Alain et al., 2015; Katharopoulos & Fleuret, 2018). Cost-aware sampling adds a square-root time correction (Mohri et al., 2026); the same form appears in Neyman allocation (Neyman, 1934; Cochran, 1977). Online samplers receive fresh information only for selected tasks, a setting studied with bandit feedback (Namkoong et al., 2017; Salehi et al., 2017; Borsos et al., 2018). The useful scale depends on the optimizer: Euclidean gradient norm fits SGD, while adaptive optimizers rescale coordinates (Lahire, 2023). GPAS applies this observation to multi-teacher MOPD and pairs it with a fixed task objective, taskwise AdamW moments, exact- $K$  task sets, and measured teacher serving time.

## 3 A FIXED OBJECTIVE AND A COMMON ADAMW STATE

Assume  $M$  teacher tasks. Let  $L_i(\theta)$  be task  $i$ ’s reverse-KL loss, averaged within each response and then over responses and prompts. Positive weights  $w_i$  sum to one and define the training objective

$$F(\theta) = \sum_i w_i L_i(\theta). \quad (1)$$

The main MOPD study uses  $L_i = \ell_i/\ell_i(0)$  and  $w_i = 1/M$ . Each initial loss  $\ell_i(0)$  is measured once at the shared starting checkpoint. This choice gives equal weight to relative improvement on each task.

A *task unit* contains a fixed number of prompts and student responses. Let  $g_i$  be its average gradient. An optimizer update selects a set  $B$  of exactly  $K$  tasks. The inclusion probability of task  $i$  is  $p_i$ , with  $\sum_i p_i = K$ . The corrected update gradient is

$$\hat{g} = \sum_{i \in B} \frac{w_i}{p_i} g_i. \quad (2)$$

Task selection happens before rollout generation. Conditional on the current student state,

$$\mathbb{E}[\hat{g}] = \sum_i w_i \mathbb{E}[g_i]. \quad (3)$$

The objective gradient is therefore fixed for every allocation rule and every  $K$ . At  $K = 1$ ,  $p_i$  is a standard task-sampling probability. At  $K = M$ , each task enters the update.

**Taskwise AdamW second moment.** Conventional AdamW inserts  $\hat{g}^2$  into its second-moment average. The expectation of this square changes with  $p_i$ . That change also alters future AdamW scaling, which makes sampler comparisons harder to interpret. We use

$$\hat{u} = \sum_{i \in B} \frac{w_i}{p_i} g_i^2, \quad (4)$$

where the square is element-wise. The first-moment average receives  $\hat{g}$ , and the second-moment average receives  $\hat{u}$ . Its conditional target is

$$\mathbb{E}[\hat{u}] = \sum_i w_i \mathbb{E}[g_i^2], \quad (5)$$

which stays fixed as  $p_i$  and  $K$  change. We call this update *taskwise AdamW*. Bias correction, learning-rate scheduling, clipping, and weight decay follow the same implementation used by the baseline. The experiments include conventional AdamW at  $K = 1$  and  $K = M$  because its different second moment can change model quality.

## 4 GPAS TASK ALLOCATION

### 4.1 OPTIMIZER-SCALED TASK SCORES

Let  $D$  be the current diagonal scaling applied by the optimizer, excluding the shared learning rate. For SGD,  $D$  is the identity. For AdamW,  $D$  comes from the current second-moment state. GPAS summarizes task  $i$  with

$$r_i^2 = \mathbb{E} \|Dg_i\|_2^2. \quad (6)$$

This score is measured from completed task units. It includes both the task’s mean gradient and sampling noise inside a task unit.

For a fixed expected task width, the part of the scaled estimator error that depends on the inclusion probabilities has the simple form

$$V(p) = \sum_i \frac{(w_i r_i)^2}{p_i}, \quad \sum_i p_i = K. \quad (7)$$

Appendix B.2 gives the calculation and the pairwise correction for exact- $K$  sets. The frozen-gradient experiment measures the complete error, so the empirical test includes that correction.

We limit every inclusion probability to  $p_{\min} \leq p_i \leq 1$ . Minimizing Equation 7 gives

$$p_i = \text{clip}\left(\frac{w_i r_i}{z}, p_{\min}, 1\right), \quad (8)$$

where the scalar  $z$  makes the probabilities sum to  $K$ . A fixed-size sampler then draws exactly  $K$  tasks with these marginals. We call this rule GPAS. The four-task study uses the maximum-entropy distribution over all possible task sets with the required marginals. Enumeration gives exact marginals and pairwise probabilities.

The warm-start diagnostic is

$$H_K = \frac{V(p^{\text{uniform}})}{V(p^{\text{GPAS}})}. \quad (9)$$

$H_K$  is a local estimator-error ratio. A value near one predicts little room for adaptive allocation at that checkpoint. Training curves determine whether the local difference improves loss.

### 4.2 COST-GPAS AND ONLINE MEASUREMENT

Let  $t_i$  be the predicted critical-path time of task  $i$ . With one resident teacher slot, the predictor uses the current resident teacher, recent transfer tail, rollout, scoring, and backward times. The additive time model leads to

$$J(p) = \left( \sum_i p_i t_i \right) V(p). \quad (10)$$

When the probability limits are inactive, the solution is

$$p_i \propto \frac{w_i r_i}{\sqrt{t_i}}. \quad (11)$$

We solve the bounded problem directly for the four-task setting. At  $K > 1$ , the solver scores every possible task set with its predicted time under the fixed teacher order and optimizes the distribution over those sets. Importance weights use the resulting inclusion probabilities. We call this allocation Cost-GPAS. The final systems comparison uses measured GPU hours.

Selected task units update exponential averages of  $r_i^2$  and task time. A round-robin warm start initializes every task. An estimate older than  $A_{\max}$  processed task units triggers a small, counted probe. The probe runs rollout, teacher scoring, and backward and leaves the student parameters fixed. Its response count and GPU time enter the resource totals. Section 5 tests the price of these refreshes directly.

GPAS training repeats four steps: refresh an expired score, compute bounded inclusion probabilities, draw an exact- $K$  task set, and apply Equations 2 and 4. The sampler state is saved with the optimizer state for exact resumption.

## 5 EXPERIMENTS

The experiments separate calculation checks from the large-model efficiency test. The controlled study verifies the estimator and optimizer-state formulas. The four-teacher study asks whether those local gains improve MOPD training per response and per GPU hour.

### 5.1 CONTROLLED CHECKS

We first use three Gaussian task gradients in three coordinates. A fixed diagonal map changes the task ranking seen by AdamW. Each method uses the same target weights and averages 16 corrected task draws. We run 20,000 trials with a fixed seed. Black crosses in Figure 1 show the calculated values.

GPAS obtains an AdamW-scaled estimator error of 0.717 relative to Uniform, while raw gradient-norm sampling obtains 6.868. Cost-GPAS obtains a time-weighted error of 0.889. The empirical values differ from the calculated values by less than 0.5% for GPAS and Cost-GPAS. A separate experiment uses one million task draws to test Equation 5. The relative second-moment error is 0.816 for conventional AdamW and 0.00235 for taskwise AdamW. These checks support the implementation and the local predictions used by GPAS.

### 5.2 PREDICTIONS TESTED IN MOPD

Each method component has a measured test:

Table 1: Connection between the analysis and the MOPD measurements.

| Prediction                                   | Measurement                                              | Comparison                                         |
|----------------------------------------------|----------------------------------------------------------|----------------------------------------------------|
| Optimizer-scaled scores reduce local error   | Held-out frozen-gradient error at $K = 1, 2, 4$          | GPAS vs. Uniform and raw-norm sampling             |
| Larger $K$ reduces task-selection noise      | Exact- $K$ error from the same bank                      | Measured change from $K = 1$ to $K = 2$ to $K = 4$ |
| Taskwise AdamW fixes the local moment target | Taskwise and conventional square observations            | Relative target error and end-to-end ablation      |
| Cost-GPAS improves time efficiency           | Loss per response, loss per GPU hour, and teacher traces | Cost-GPAS vs. GPAS and Uniform                     |

The frozen-bank comparisons use one half of the task units to estimate the proposal and the other half to evaluate it. Swapping the halves uses all units while keeping proposal construction separate from evaluation. Exact subset enumeration supplies the pairwise probabilities for  $K = 2$ .

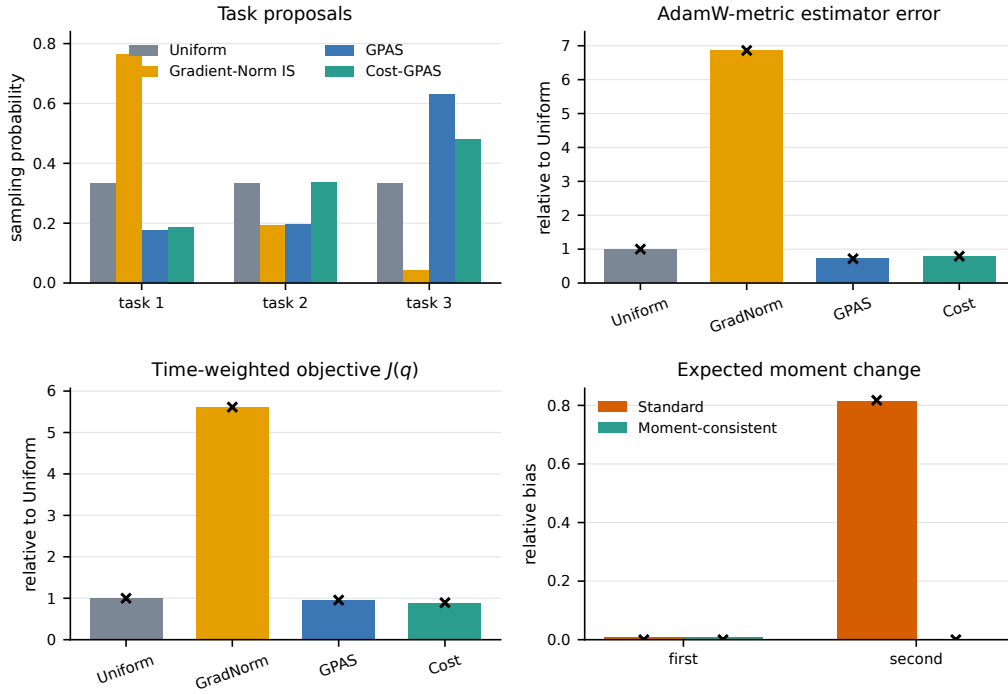


Figure 1: Controlled checks. GPAS follows gradient scale after AdamW scaling; Cost-GPAS also uses task time. Bars are Monte Carlo estimates and crosses are calculated values. All ratios use Uniform as the reference.

### 5.3 FOUR-TEACHER SETUP

The student starts from a fixed Qwen3-1.7B revision (Qwen Team, 2025). Four fixed specialists provide mathematics, code, instruction-following, and science feedback. The run manifest records the exact model and tokenizer revisions, precision, sharding, data versions, prompt templates, hardware, and software. The teacher set and student initialization remain fixed for every method.

One task unit contains 16 prompts and four independently sampled responses per prompt, for 64 attempted responses. Reverse KL is averaged over valid tokens in each response, over the four responses for a prompt, and then over prompts. The response cap is 8,192 tokens. Every attempted response enters the response and resource totals. The main objective is

$$F_{\text{rel}}(\theta) = \frac{1}{M} \sum_i \frac{\ell_i(\theta)}{\ell_i(0)}. \quad (12)$$

The initial denominators and evaluation prompts are frozen before method comparisons.

Every run starts from a shared round-robin warm-start checkpoint. Each task contributes two complete task units to that checkpoint. The all-in budget is 128,000 attempted responses, including probes. Learning-rate scheduling uses attempted responses as its clock. A  $K$ -task update uses  $\beta_1^K$  and  $\beta_2^K$  when  $\beta_1$  and  $\beta_2$  denote one-task-unit AdamW decay. The appendix specifies the matching weight-decay conversion and resume test.

### 5.4 TASK WIDTHS AND BASELINES

Table 2 includes the intermediate width required to test the task-width prediction. It also separates sampling changes from AdamW-state changes.

Table 2: MOPD configurations. TW denotes the taskwise second moment in Equation 4; Conv. denotes conventional AdamW.

| Allocation                   | $K$ | Inclusion rule        | AdamW state |
|------------------------------|-----|-----------------------|-------------|
| Uniform baseline             | 1   | uniform               | Conv.       |
| Uniform state control        | 1   | uniform               | TW          |
| GPAS                         | 1   | Eq. 8                 | TW          |
| Cost-GPAS                    | 1   | Eq. 11                | TW          |
| Uniform intermediate         | 2   | uniform exact set     | TW          |
| Cost-GPAS intermediate       | 2   | bounded Cost-GPAS set | TW          |
| Streamed mixed state control | 4   | all tasks             | TW          |
| Streamed mixed baseline      | 4   | all tasks             | Conv.       |

All configurations use matched per-task prompt streams: the  $n$ th request for each task receives the same prompt group across configurations. This pairing isolates the sampler and optimizer-state mechanisms.

### 5.5 ONE TEACHER SLOT AND COMPLETE COST ACCOUNTING

$K$  counts task units combined before one optimizer update. Teacher residency is a separate system choice. Every configuration uses the same GPU type, GPU count, and one resident teacher slot. A  $K = 4$  update visits four teachers in a fixed order and accumulates their gradients at one student state. The resident teacher is served first when it belongs to the selected set. The final teacher stays resident for the next update.

Transfers may overlap student rollout or backward. Each task unit records teacher load and offload times, teacher-ready time, rollout, scoring, backward, optimizer time, peak HBM, response count, and token count. Transfer tail is  $\max(0, t_{\text{ready}} - t_{\text{rollout end}})$ . GPU hours equal wall time times the number of reserved GPUs, including transfer and waiting periods. These traces test whether reduced estimator error repays teacher-switch and probe costs.

### 5.6 OUTCOMES AND UNCERTAINTY

The two primary plots show mean relative teacher loss against cumulative attempted responses and GPU hours. The main table reports final mean and task-level relative loss, responses and GPU hours to a common loss threshold, peak HBM, responses and tokens per GPU hour, switch rate, transfer tail, and update-time quantiles. A warm, middle, and late checkpoint from the Uniform trajectory each supply eight frozen task units per task for the mechanism tests in Table 1.

Held-out loss intervals use paired prompt bootstrap with fixed evaluation generation seeds. Training plots show each configuration under the matched prompt streams. Capability evaluation reports MATH-500 greedy pass@1, a frozen LiveCodeBench pass@1 slice, IFBench strict accuracy, and GPQA-Diamond average@4 alongside the teacher-loss efficiency results.

The online study fixes  $A_{\text{max}} = 50$  processed task units. Logs report score age, probe count, probe GPU hours, final bounded inclusion probabilities, and importance multipliers. The EMA decay, inclusion floor, failure penalty, and common loss threshold are frozen in the run manifest before training results are viewed.

## 6 LIMITATIONS AND CONCLUSION

GPAS uses a local view of training. The current AdamW scaling, task gradients, and task times can change as the student learns. Bounded-age probes expose the cost of refreshing these measurements. Frozen banks at three checkpoints show whether the allocation score remains predictive during training.

The simple allocation objective prices first-order inclusion probabilities. An exact- $K$  task set also depends on pairwise task inclusion and gradient alignment. Four-task enumeration lets us measure this term directly. The water-filled probabilities serve as a compact allocation rule, while the held-out exact-set error supplies the empirical check.

Taskwise AdamW creates a proposal-independent local second-moment target. It is a different optimizer-state update from conventional AdamW and can change the effective step size. The end-point baselines and capability evaluations measure that practical difference.

Cost-GPAS uses recent critical-path measurements and the current resident teacher. Transfer contention and cache state can still create prediction error. End-to-end GPU hours and teacher traces decide the systems result. The additive time model serves as the online allocation score.

The central MOPD design principle is simple: state the teacher-loss objective, keep its weights fixed during method comparison, measure gradients in the optimizer’s local geometry, and charge every response and teacher switch. GPAS implements this principle for a chosen task width. Cost-GPAS adds measured serving time. The  $K = 1, 2, 4$  study determines when local estimator gains translate into faster MOPD training.

#### AI USE STATEMENT

We used generative AI tools for language editing and manuscript organization. The authors reviewed every assisted revision and take responsibility for the text, claims, proofs, code, and experimental results.

#### ETHICS STATEMENT

The study evaluates optimization methods for existing language models with reasoning and instruction-following datasets and automated verifiers. Model and dataset biases can affect capability scores. We record data provenance, invalid outputs, and verifier failures to support careful interpretation.

#### REPRODUCIBILITY STATEMENT

Section 5 specifies task units, objective normalization, exact- $K$  sampling, teacher streaming, resource accounting, baselines, and matched evaluation streams. The released artifact includes controlled-check code and seeds, the run manifest, model and tokenizer revisions, objective denominators, prompt streams, optimizer and sampler states, inclusion and probe logs, transfer traces, per-prompt evaluation output, and source data for every figure and table.

#### REFERENCES

- Guillaume Alain, Alex Lamb, Chinnadhurai Sankar, Aaron Courville, and Yoshua Bengio. Variance reduction in sgd by distributed importance sampling, 2015. URL <https://arxiv.org/abs/1511.06481>.
- Alon Albalak, Liangming Pan, Colin Raffel, and William Yang Wang. Efficient online data mixing for language model pre-training, 2023. URL <https://arxiv.org/abs/2312.02406>.
- Zalan Borsos, Andreas Krause, and Kfir Y. Levy. Online variance reduction for stochastic optimization. In *Proceedings of the 31st Conference on Learning Theory*, pp. 324–357, 2018. URL <https://proceedings.mlr.press/v75/borsos18a.html>.
- Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 794–803, 2018. URL <https://proceedings.mlr.press/v80/chen18a.html>.
- William G. Cochran. *Sampling Techniques*. Wiley, 3 edition, 1977.
- Huan-ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-mopd: Diagnosing and fixing capability imbalance in multi-teacher on-policy distillation, 2026. URL <https://arxiv.org/abs/2608.19098>.

- Zhaolin Gao, Joongwon Kim, Wen Sun, Thorsten Joachims, Sid Wang, Richard Yuanzhe Pang, and Liang Tan. Prompt curriculum learning for efficient LLM post-training, 2025. URL <https://arxiv.org/abs/2510.01135>.
- Angelos Katharopoulos and François Fleuret. Not all samples are created equal: Deep learning with importance sampling. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 2525–2534, 2018. URL <https://proceedings.mlr.press/v80/katharopoulos18a.html>.
- Vitaly Kurin, Alessandro De Palma, Ilya Kostrikov, Shimon Whiteson, and M. Pawan Kumar. In defense of the unitary scalarization for deep multi-task learning. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Thibault Lahire. Non-uniform sampling and adaptive optimizers in deep learning. OPT 2023: Optimization for Machine Learning, 2023. URL <https://opt-ml.org/papers/2023/paper20.pdf>.
- Zhaoyi Li, Deyang Kong, Yuan Wei, Evan Yang, Ranran Shen, Mahardika Krisna Ihsani, Ming Yang, Wei Zhang, Chuan Hao, Jian Yang, Ran Tao, Bryan Dai, Shikun Zhang, Wei Ye, Ying Wei, and Defu Lian. Every coin has two sides: On the dual nature of generalization in on-policy distillation of large language models, 2026. URL <https://arxiv.org/abs/2608.16647>.
- Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning, 2024. URL <https://arxiv.org/abs/2110.14048>.
- Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. Mopd: Multi-teacher on-policy distillation for capability integration in llm post-training, 2026. URL <https://arxiv.org/abs/2606.30406>.
- Clara Mohri, Amir Globerson, Haim Kaplan, Tomer Koren, and Yishay Mansour. Cost-aware learning, 2026. URL <https://arxiv.org/abs/2604.28020>.
- Hongseok Namkoong, Aman Sinha, Steve Yadlowsky, and John C. Duchi. Adaptive sampling probabilities for non-smooth optimization. In *Proceedings of the 34th International Conference on Machine Learning*, pp. 2574–2583, 2017. URL <https://proceedings.mlr.press/v70/namkoong17a.html>.
- Jerzy Neyman. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97(4): 558–625, 1934. doi: 10.2307/2342192.
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Farnood Salehi, L. Elisa Celis, and Patrick Thiran. Stochastic optimization with bandit sampling, 2017. URL <https://arxiv.org/abs/1708.02544>.
- Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization, 2019. URL <https://arxiv.org/abs/1810.04650>.
- Zeichen Sun, Zhiwei Zhang, Fei Zhao, Juntao Li, Mu Chuan, Huayu Deng, Guojian Zhan, Wenliang Chen, Yao Hu, and Min Zhang. D<sup>3</sup>-mopd: Adaptive dynamic domain scheduling for efficient multi-teacher distillation, 2026. URL <https://arxiv.org/abs/2608.24987>.
- Zhenting Wang, Guofeng Cui, Kun Wan, and Wentian Zhao. DUMP: Automated distribution-level curriculum learning for RL-based LLM post-training, 2025. URL <https://arxiv.org/abs/2504.09710>.
- Derrick Xin, Behrooz Ghorbani, Justin Gilmer, Ankush Garg, and Orhan Firat. Do current multi-task optimization methods in deep learning even help? In *Advances in Neural Information Processing Systems*, volume 35, 2022.



Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. URL <https://arxiv.org/abs/2001.06782>.

Haizhong Zheng, Yang Zhou, Brian R. Bartoldson, Bhavya Kailkhura, Fan Lai, Jiawei Zhao, and Beidi Chen. Act only when it pays: Efficient reinforcement learning for LLM reasoning via selective rollouts, 2025. URL <https://arxiv.org/abs/2506.02177>.

## A PRACTICAL TRAINING RECIPE

**1. Fix the objective.** Define a task unit with fixed prompt and response counts. Average token loss within each response. Measure each initial task loss at the shared checkpoint and freeze the normalized objective weights.

**2. Profile the candidate task widths.** Run a round-robin warm start. Measure optimizer-scaled gradient magnitude and critical-path time for every task. Compute  $H_K$  for each candidate  $K$  and record memory and transfer traces.

**3. Draw an exact- $K$  set.** Compute bounded GPAS or Cost-GPAS inclusion probabilities. Draw exactly  $K$  tasks with those marginals. Add  $w_i g_i / p_i$  to the gradient accumulator and  $w_i g_i^2 / p_i$  to the taskwise second-moment accumulator as each unit finishes.

**4. Stream teachers.** Use the same residency policy for every  $K$ . With one slot, serve the resident teacher first, prefetch the next teacher when memory permits, and keep the final teacher resident. Record every transfer phase and the end-to-end critical path.

**5. Refresh and save sampler state.** Run a counted probe when a score reaches the age limit. Save probabilities, selected sets, score and time averages, ages, probe state, response counts, transfer state, and both optimizer accumulators. A resume test reproduces the next set and optimizer update.

## B DERIVATIONS AND EVALUATION DETAILS

### B.1 IMPORTANCE CORRECTION AND TASKWISE MOMENTS

Let  $Z_i$  indicate that task  $i$  enters the selected set. Selection precedes generation, so  $Z_i$  and the potential task-unit gradient  $g_i$  are conditionally independent. Since  $\mathbb{E}[Z_i] = p_i$ ,

$$\mathbb{E}\left[\sum_i Z_i \frac{w_i}{p_i} g_i\right] = \sum_i w_i \mathbb{E}[g_i]. \quad (13)$$

Replacing  $g_i$  with its element-wise square gives

$$\mathbb{E}\left[\sum_i Z_i \frac{w_i}{p_i} g_i^2\right] = \sum_i w_i \mathbb{E}[g_i^2]. \quad (14)$$

Both identities hold for any fixed-size design with the stated marginals.

At  $K = M$ , write  $\bar{g} = \sum_i w_i g_i$ . The difference between taskwise and conventional mixed-batch square observations is

$$\sum_i w_i g_i^2 - \bar{g}^2 = \sum_i w_i (g_i - \bar{g})^2 \geq 0 \quad (15)$$

coordinate by coordinate. This task-dispersion term explains why the two AdamW variants can use different effective step sizes.

## B.2 ESTIMATOR ERROR FOR FIXED-SIZE TASK SETS

Let  $h_i = \mathbb{E}[g_i]$ ,  $h = \sum_i w_i h_i$ , and  $e_i = \mathbb{E}\|D(g_i - h_i)\|_2^2$ . An independent-inclusion calculation gives

$$\mathbb{E}\|D(\hat{g} - h)\|_2^2 = \sum_i w_i^2 \left[ \frac{1-p_i}{p_i} \|Dh_i\|_2^2 + \frac{e_i}{p_i} \right]. \quad (16)$$

Since  $r_i^2 = \|Dh_i\|_2^2 + e_i$ , the part controlled by  $p$  equals  $V(p)$  plus a constant.

For an exact- $K$  set, let  $p_{ij}$  be the probability that tasks  $i$  and  $j$  enter together. The complete error adds

$$2 \sum_{i < j} w_i w_j \left( \frac{p_{ij}}{p_i p_j} - 1 \right) \langle Dh_i, Dh_j \rangle. \quad (17)$$

This term depends on task alignment and the subset design. The experiment enumerates every subset for four tasks and evaluates Equations 16–17 together.

## B.3 GPAS AND COST-GPAS SOLUTIONS

Write  $a_i = w_i r_i$ . For  $V(p) = \sum_i a_i^2 / p_i$ , the interior optimality condition gives  $p_i = a_i / z$ . Coordinates that reach either probability limit stay at that limit, and the same condition applies to the remaining coordinates. This is Equation 8.

For task times  $t_i > 0$ , Cauchy–Schwarz gives

$$\left( \sum_i p_i t_i \right) \left( \sum_i \frac{a_i^2}{p_i} \right) \geq \left( \sum_i a_i \sqrt{t_i} \right)^2. \quad (18)$$

Equality holds at  $p_i \propto a_i / \sqrt{t_i}$ , which gives Equation 11. The implementation numerically solves the small constrained problem when a probability limit is active.

## B.4 MOPD GRADIENT AND PROBE SCALING

For a student-generated prefix  $z$ , write  $q_v = \pi_\theta(v \mid z)$  and  $t_v = T_i(v \mid z)$ . The sampled-token reverse-KL score is

$$\text{sg} \left[ \log \frac{q_V}{t_V} \right] \nabla_\theta \log q_V, \quad V \sim q. \quad (19)$$

The implementation averages token contributions within a response and then over the fixed task unit. All allocation claims use this conditional, rollout-fixed surrogate gradient.

The full task unit averages 16 prompt groups. An eight-response probe contains two groups. For task mean  $h_i$  and prompt-group noise  $b_i^2$ , the full-unit score is

$$r_i^2 = \|Dh_i\|_2^2 + b_i^2 / 16. \quad (20)$$

The probe refreshes the maintained mean and group-noise estimates. Probe time is tracked in its own resource counter.

## B.5 RESPONSE-CLOCK ADAMW AND SYSTEM LOGS

If  $\beta$  denotes the AdamW decay after one task unit, a  $K$ -task combined update uses  $\beta^K$ . This matches the total decay assigned to  $K$  processed units. Decoupled weight decay uses the matching compounded coefficient. A unit test compares the decay of stored moments and parameters against  $K$  repeated clock advances. A resume test reproduces the next selected set, task order, moment observation, and parameter update.

Every checkpoint stores  $K$ , inclusion probabilities, selected set and order, scale and time averages, observation ages, probes, resident teacher, and AdamW variant. Each task unit logs prompt, response, and token counts; gradient norm, clipping, and overflow; transfer phases; rollout, scoring, backward, and optimizer time; GPU count; and peak HBM. Frozen-bank source units, proposals, and held-out errors are released with the run data.